

Question 2.1: The Robustness Duel

Comparing FCNN and CNN Performance on Shifted MNIST

February 14, 2026

1 Introduction

This report investigates the robustness of two different neural network architectures when tested on spatially transformed images. We compare a Fully Connected Neural Network (FCNN) and a Convolutional Neural Network (CNN) on their ability to maintain accuracy when input images undergo spatial shifts.

1.1 Objective

The objective of this experiment is to understand how spatial transformations affect different network architectures and determine which design is more robust to position changes in input data.

2 Models

2.1 Model A: Fully Connected Neural Network (FCNN)

The FCNN architecture consists of:

- Input layer: 784 neurons (28×28 flattened image)
- Hidden layer 1: 128 neurons with ReLU activation
- Hidden layer 2: 128 neurons with ReLU activation
- Hidden layer 3: 128 neurons with ReLU activation
- Output layer: 10 neurons (digit classes 0-9)
- Total parameters: 109,386

2.2 Model B: Convolutional Neural Network (CNN)

The CNN architecture consists of:

- Convolutional Layer 1: 32 filters with 3×3 kernel, padding=1, ReLU activation
- Max Pooling 1: 2×2 kernel, stride=2
- Convolutional Layer 2: 64 filters with 3×3 kernel, padding=1, ReLU activation
- Max Pooling 2: 2×2 kernel, stride=2
- Fully Connected Layer 1: 128 neurons with ReLU activation
- Output layer: 10 neurons (digit classes 0-9)
- Total parameters: 409,098

3 Training Parameters

Both models were trained using the following configuration:

- Dataset: MNIST handwritten digits
- Training samples: 60,000 images
- Test samples: 10,000 images
- Image size: 28×28 pixels (grayscale)
- Optimizer: Adam
- Learning rate: 0.001
- Batch size: 128
- Number of epochs: 10
- Loss function: Cross-entropy loss

The robustness test was conducted by creating a "Shifted MNIST" test set, where every image from the original test set was translated 4 pixels to the right.

4 Visualization

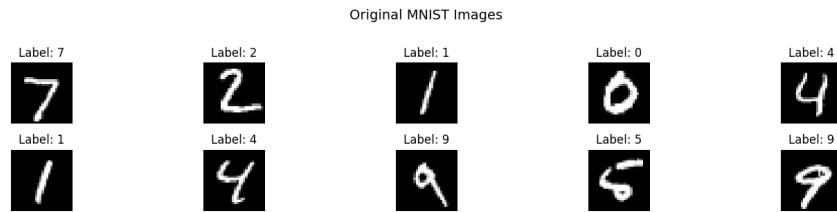


Figure 1: Sample images from the original MNIST test set

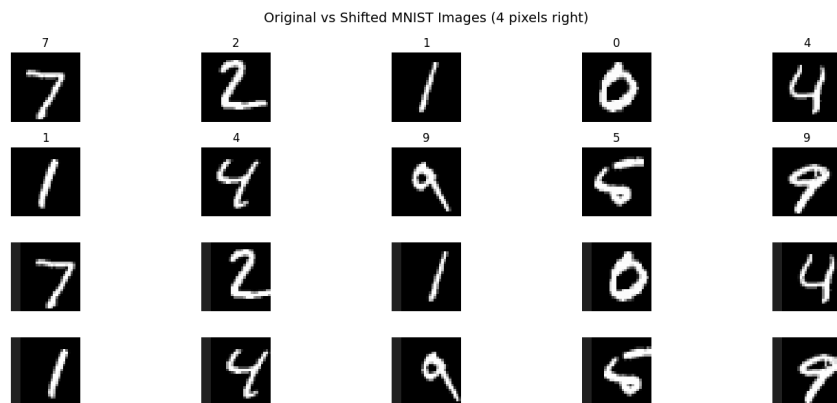


Figure 2: Comparison of original (rows 1-2) and shifted images (rows 3-4). Each image is shifted 4 pixels to the right.

5 Results

Table 1 presents the accuracy of both models on the original test set and the shifted test set. Figure 3 provides a visual comparison of the performance metrics.

Table 1: Model Performance Comparison

Model	Original Test	Shifted Test	Accuracy Drop
FCNN (Model A)	97.3%	35.2%	62.04%
CNN (Model B)	99.0%	72.6%	26.46%

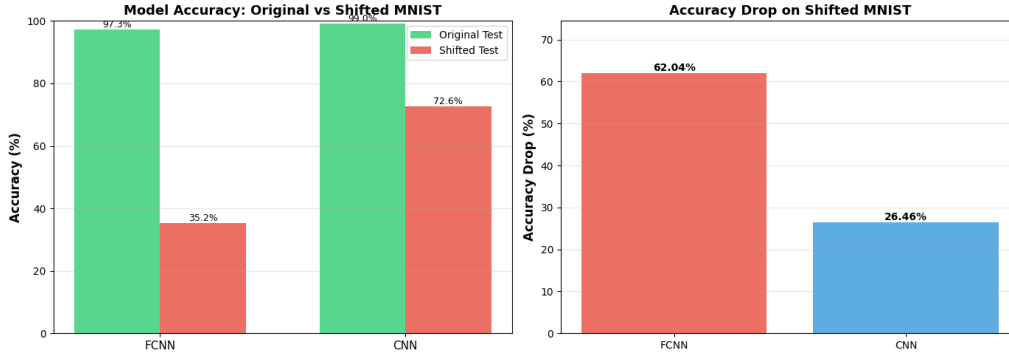


Figure 3: Left: Model accuracy on original vs shifted test sets. Right: Accuracy drop when images are shifted 4 pixels to the right.

5.1 Key Observations

- Both models achieve high accuracy on the original test set (FCNN: 97.3%, CNN: 99.0%)
- FCNN experiences a severe accuracy drop of 62.04% on shifted images
- CNN maintains much better robustness with only a 26.46% accuracy drop
- CNN is approximately 2.3 times more robust than FCNN to spatial shifts

6 Thoughtful Analysis

6.1 Why CNN Maintains Higher Accuracy Despite the Shift

The large difference in robustness between the two models is mainly because of the **weight sharing property** used in convolution.

FCNN - Position-Specific Learning:

In an FCNN, each neuron learns separate weights for every pixel in the image. This means it learns features based on exact positions. For example, if it learns a pattern at one location, it will only recognize it there. If the image shifts slightly, the pattern moves to a new position, and the neuron may fail to detect it. This makes FCNNs sensitive to small shifts.

CNN - Weight Sharing and Translation Equivariance:

CNNs use the same filter across the whole image instead of different weights for each position. The filter slides over the image and looks for the same pattern everywhere. Because of this, even if the image shifts, the model can still detect the feature in its new position.

This property is called **translation equivariance**. It means that when the input shifts, the detected features also shift but are still recognized. So a filter trained to detect something like an edge can find it anywhere in the image, not just in one fixed spot.